SimpleDB Lab 2 Report

Design Decisions

1. LRU

We maintain the LRU order with a linked list. Whenever a page is queried, there are two possibilities – 1: Page already exists, and 2: Page is new.

If the page already exists, we remove it from the linked list and append it to the end, such that it is now the least prioritized for removal since it is the most recently used.

If the page does not exist (is new), we read it from the disk, add it to the HashMap, and then add it to the end of the linked list. Eviction is handled separately to allow the code to be more modular and suitable for handling new or empty buffer pools.

2. Eviction

When the pool is full, we first remove a page's ID from the LRU tracking linked list above using the function removeFirst() (Java method for LinkedList class). With the LRU order being maintained by the function above, the page removed by removeFirst() always clears the least recently used page. We then store the pid of the removed page and use it in the next method, flushPage()

3. Flush Page

In flushPage(), we check if the list is dirty. If it is, we use the Catalog class to write the page back to the database. If it's not, we skip the operation to reduce disk I/O.

flushAllPages() is an assignment requirement and simply flushes every page without removing them from the pool.

4. Insert / Delete

The insert and delete functions iterate through all the tuples provided by their children operator (OpIterator[]) and delegate each insertion/deletion task to the Database.getBufferPool() method (BufferPool class). (Encapsulation)

After the operations are done, a tuple is returned containing the amount of affected records.

Separately, in the BufferPool.java class, insertion and deletion operations are implemented there with the additional logic of marking the associated page as dirty to be used in the above flushPage() logic.

In the HeapFile, new logic is added. Most critically, HeapFile.insertTuple() has the logic of searching for an empty slot in each page, and if it fails, it appends a new page for the new tuple. For .deleteTuple(), the page id is retrieved directly from the tuple's RecordId to avoid an expensive sequential search across the entire file.

HeapPage implementation:

Insertions and deletions have the added functionality of marking a slot as occupied or unoccupied and giving a record id for a create operation and clearing the slot for a delete operation.

The occupied status is also implemented with a bitmap instead of an array of Boolean values, allowing for a more memory-efficient design and increasing the amount of records that can be stored per page, thus decreasing average lookup time.

Aggregation implementation:

The aggregation implementation is split across two important files, Aggregate.java and IntegerAggregator.java.

In Aggregate.java, open() is called, sequentially reading all provided tuples and calling mergeTupleIntoGroup() for each one. The implementation of mergeTupleIntoGroup() is in IntegerAggregator.java, where for each tuple, it determines its group (or lack thereof), extracts the aggregate field and updates it according to the group by field (if there is no group by field, then it is part of a large group known as NO_GROUPING_FIELD.

StringAggregator is also implemented, though it only implements COUNT as the other aggregation options cannot be performed on strings. It behaves similarly to IntegerAggregator.